
ENGINEERING PLAYBOOK

EdgeLM

How the shared on-device AI runtime works

A concept-by-concept tour of the runtime engine, memory model, scheduler, IPC contract and API surface
— with internal source pointers and external references for every idea.

Android · on-device · llama.cpp · Binder/AIDL · Q4_0 GGUF
Audience: engineers & architects · v0.1 · July 2026

What's inside

- 01 The big idea — one model, every app
- 02 System architecture — the shape of the runtime
- 03 The IPC contract — AIDL, Binder & the token callback
- 04 The runtime engine — llama.cpp behind a JNI boundary
- 05 Memory architecture — mmap, sharing & idle unload
- 06 Model catalog & quantization — why Q4_0 imatrix
- 07 Sessions & the KV cache — cheap multi-turn
- 08 The AI scheduler — priority, aging, non-preemption
- 09 Speculative decoding — and why it's GPU-only
- 10 The two front doors — Binder SDK & OpenAI HTTP shim
- 11 Security & permission model
- 12 The developer SDK — Flow in a few lines
- 13 Glossary & full reference index

HOW TO READ THIS

Each concept follows the same rhythm: **what it is** and why it matters on a phone, **how EdgeLM does it** (with the exact source file), and a **reference strip** pairing an authoritative external source with the internal file or doc. Blue underlined text is a live link.

SECTION 01

The big idea

One model in memory, shared by every app on the phone.

Every app that wants on-device AI today ships and loads its *own* copy of a model. Three such apps on a 6–8 GB phone means three multi-gigabyte weight blobs fighting for RAM against the OS and the foreground app — and the Android low-memory killer is always watching. EdgeLM's thesis is that the model is **system infrastructure**, not app baggage.

EdgeLM is a standalone runtime app (installed once from Google Play) that holds the model in memory and serves generations to any number of client apps. Download a model once; a second client app adds almost no extra memory because it reuses the same resident weights. Prompts and responses never leave the device, and once a model is downloaded the runtime works fully offline.

THE FOUR PROPERTIES EVERYTHING ELSE SERVES

Private — inference is local; no cloud, account, or telemetry on the prompt path.

Offline — after download, zero network dependency.

Shared — one mmap'd model serves all apps; the marginal app is nearly free.

Efficient — no per-request cost, no network round-trips, memory released when idle.

■ REFERENCES

Product vision: [README.md — "Why EdgeLM"](#) (internal) · architecture doc Part 01

On-device AI, background: [Google AI Edge / on-device inference overview](#) (external)

The RAM wall: [Android low-memory killer \(LMK\) daemon](#) (external)

SECTION 02

System architecture

Two front doors, one inference path, one process.

The runtime is a single Android *foreground service* living in its own process (:core). It exposes **two front doors** that both funnel into one serialized inference path over a single warm model:

1. Binder (AIDL) — the native path used by the EdgeLM SDK. It carries the caller's UID, so the runtime always knows which app is asking.

2. A localhost HTTP shim — an OpenAI-compatible server on `127.0.0.1:1408` so existing OpenAI SDKs and cURL work unmodified. It runs *inside* the service process, so it calls the engine directly with no second Binder hop.

Underneath, one `llama_context` is warm at a time, so exactly one generation runs at a time. The **AI scheduler** (Section 08) decides which waiting request goes next. A persistent notification shows the runtime is active, lets the user free memory on demand, and the runtime auto-releases the model after five idle minutes while the service itself stays reachable.

REQUEST LIFECYCLE (BINDER PATH)

app calls `EdgeLM.chat()` → SDK binds the service → `submit()` over Binder with a token callback → scheduler admits the job by priority → JNI `generate()` streams token chunks back through the oneway callback → `onDone(tokenCount, elapsedMs)`.

■ REFERENCES

Service & both front doors: `EdgeLMService.kt` (internal) · architecture doc Part 02

Android services: [Foreground services — Android developers](#) (external)

Process isolation: [Android application sandbox](#) (external)

SECTION 03

The IPC contract

AIDL, Binder, and a one-way streaming callback.

Because clients and the runtime are separate apps in separate processes, they talk over **Binder**, Android's kernel-mediated IPC. The wire shape is declared in **AIDL** (Android Interface Definition Language), which generates the proxy/stub marshalling code. The whole contract lives in the shared `:contract` module so both sides compile against the identical interface.

The service interface is deliberately tiny:

```
interface IEdgeLMService {
    long submit(String model, String sessionId, String prompt,
               int priority, ITokenCallback callback); // -> requestId
    void cancel(long requestId); // frees the decode loop + its KV
    String[] warmModels(); // which models are loaded now
    boolean reloadModel(); // (re)load model.gguf after download
    boolean unloadModel(); // reclaim RAM; lazy-reloads next call
    String prepareEngine(); // warm + run CPU-vs-GPU probe
}
```

Tokens flow back over a separate **oneway** callback so the runtime never blocks on a slow client while decoding — it fires and forgets each chunk:

```
oneway interface ITokenCallback {
    void onTokens(String chunk); // 1+ tokens, batched to limit IPC
    void onDone(int tokenCount, long elapsedMs); // -> tokens/sec
    void onError(String message);
}
```

ABI DISCIPLINE

New methods (`reloadModel`, `unloadModel`, `prepareEngine`) are **appended** to the interface, never reordered. Binder dispatches by method index, so inserting a method in the middle would silently break every already-installed client.

■ REFERENCES

Contract source: `contract/src/main/aidl/ai/edgelm/contract/*.aidl` (internal)

AIDL: [Android Interface Definition Language \(AIDL\)](#) (external)

Binder IPC: [Bound services / Binder overview](#) (external)

SECTION 04

The runtime engine

llama.cpp, reached through a thin JNI boundary.

The actual inference is **llama.cpp** compiled for ARM and loaded as a native library (`libedge_lm.so`). Kotlin never touches the model directly; it crosses a small, stable **JNI** seam — `NativeBridge` — whose whole job is to be the one place C++ meets the JVM. The boundary is intentionally narrow so the native decode loop can change without disturbing anything above it.

```
object NativeBridge {
    external fun loadModel(path: String): Long          // mmap GGUF -> handle
    external fun generate(handle, sessionId, prompt, sink): Int
    external fun cancel(handle: Long)                   // cooperative cancel
    external fun unloadModel(handle: Long)              // munmap + free
    external fun engineLabel(): String                  // "CPU" / "GPU . Mali-G615"
    external fun attachDraft(handle, draftPath): Boolean // speculative decoding
}
```

Generation is **blocking and off the main thread**: C++ pushes text into a `TokenSink` callback, and on every chunk it also checks `isCancelled()` so a cancelled request stops promptly and frees its work. The build targets `armv8.2-a+dotprod+i8mm+fp16` so `llama.cpp` can use the fast integer matrix-multiply kernels the quantized weights are designed for.

WHY A HARD JNI LINE

In Phase 0 the native side even streamed placeholder tokens while the Kotlin service, scheduler and IPC were built and tested against the real interface. Swapping in the true `llama.cpp` decode changed nothing above `NativeBridge.generate()` — that is the entire point of the boundary.

■ REFERENCES

JNI bridge source: `runtime-service/.../service/NativeBridge.kt` (internal)

Inference core: [llama.cpp \(ggml-org\)](#) (external)

JNI: [Java Native Interface specification](#) (external)

ARM kernels: [Arm KleidiAI / i8mm+dotprod matmul](#) (external)

SECTION 05

Memory architecture

On a phone the wall is RAM, not FLOPs.

A mid-range phone shares 6–8 GB across the OS, the foreground app and everything else. EdgeLM's memory manager exists to run the largest possible model at the highest throughput while staying *invisible* to the low-memory killer. The foundation is **mmap**: weights are never read () into a heap buffer — the GGUF file is memory-mapped, so the kernel page cache *is* the model's memory.

WHAT MMAP BUYS

Zero-copy, instant load — pages fault in on demand, no multi-second copy.

Sharing across UIDs — the file's pages are backed once; the shared runtime means every client app is served from that single resident copy.

Graceful reclaim — clean file-backed pages can be dropped by the kernel under pressure and faulted back later, instead of getting the process killed.

On top of that, the service is a **foreground service** (so the OS treats it as in-use and OEM "freezers" leave it alone) and it **auto-unloads the model after 5 idle minutes**. Unloading frees the RAM but keeps the service alive and reachable; the next request lazily re-mmaps the model. The user can also free memory on demand from the notification.

■ REFERENCES

Memory manager: `EdgeLMService.kt` (idle unload / foreground) (internal) · arch doc Part 04

mmap: [mmap\(2\) — Linux manual page](#) (external)

GGUF mmap in llama.cpp: [llama.cpp memory/mmap notes](#) (external)

LMK: [Low memory killer daemon \(lmkd\)](#) (external)

SECTION 06

Model catalog & quantization

Why every model ships as Q4_0 imatrix GGUF.

The runtime app presents a curated catalog spanning "runs on anything" to "flagship only", each entry described in plain language with size, context window and a RAM warning. All entries are 4-bit **Q4_0 GGUF** files, calibrated with an **importance matrix (imatrix)**. This is a deliberate optimum for ARM CPUs, not a compromise.

In-app name	Model	Good for
Quick Assistant	Qwen2.5 0.5B	Fast, light — quick questions on any phone
Everyday Assistant	Llama 3.2 1B	Chatting, writing, summarizing
Smart Assistant	Qwen2.5 1.5B	Sharper answers, better multilingual
Pro Assistant	Llama 3.2 3B	Higher-quality writing and thinking
Expert Assistant	Phi-3.5 mini	Tricky questions, math, coding help

Why Q4_0 + imatrix wins on ARM CPU. Modern llama.cpp *repacks* a plain Q4_0 tensor into an i8mm/dotprod-optimized layout **at load time**, giving the fastest ARM decode path for free. imatrix calibration weights which parameters matter during quantization, so quality-per-bit is better at the same 4-bit size. The tempting alternatives are worse for this target: IQ4_XS and Q4_K_M miss the repack path and decode slower on CPU; Q3/IQ3 lose too much quality.

THE HARD CONSTRAINT

CPU decode is **memory-bandwidth-bound**: each token reads (nearly) the whole weight tensor, so tok/s at a fixed quality is set by bytes-loaded-per-token. Measured on the target SoC: 3B Q4_0 ≈ 7.5–8.5 tok/s, 0.5B ≈ 25 tok/s. You can't make "the 3B at 3B quality" meaningfully faster on CPU — every real lever is quality-per-byte or moving off CPU.

■ REFERENCES

Catalog + findings: ModelCatalog.kt · docs/MODEL-OPTIMIZATION-FINDINGS.md (internal)

GGUF format: [GGUF spec \(ggml-org\)](#) (external)

Quantization types: [llama.cpp quantization overview](#) (external)

imatrix: [Importance-matrix quantization \(llama.cpp\)](#) (external)

SECTION 07

Sessions & the KV cache

Multi-turn that doesn't re-read the whole conversation.

A transformer generating token N attends to keys and values for tokens $1..N-1$. Recomputing those every step would be quadratic, so they're cached — the **KV cache**. That cache is also what makes cheap multi-turn possible, and it's why `submit()` takes a `sessionId`.

HOW SESSIONID BEHAVES

"" (empty) — stateless one-shot; the KV is built for this prompt and dropped.

stable id — multi-turn. Prior turns stay warm in the runtime's KV cache and are **not re-prefilled**, so a follow-up only pays for its new tokens instead of re-reading the whole history. `cancel()` frees the decode loop and its KV.

```
EdgeLM.chat("default", "My name is Ada.", sessionId = "s1").collect { ... }  
EdgeLM.chat("default", "What's my name?", sessionId = "s1").collect { ... } // remembers
```

Because prefill (reading the prompt) is compute-heavy and decode is bandwidth-heavy, skipping prefill of the conversation history is the single biggest win for snappy follow-ups. The trade-off is memory: each live session holds its KV, which is why the contract exposes explicit cancel to release it. (KV-cache *quantization* was evaluated and shelved — at short context the KV is a few MB, negligible next to the ~1.9 GB of weights read per token.)

■ REFERENCES

Session semantics: [IEdgeLMService.aidl doc · INTEGRATION.md "Multi-turn"](#) (internal)

KV cache explained: [The KV cache / transformer inference \(Hugging Face\)](#) (external)

llama.cpp KV: [llama.cpp KV cache & context](#) (external)

SECTION 08

The AI scheduler

One engine, many callers, no starvation.

There is exactly one warm `llama_context`, so only one generation runs at a time. When multiple apps ask at once, `AI Scheduler` decides who goes next. Each request declares a priority; the scheduler admits the highest *effective* priority when the engine frees.

Class	API	Base	Meaning
FOREGROUND	3	1000	User is staring at the result
INTERACTIVE	2	700	Default
BATCH	1	300	Bulk / deferred work
BACKGROUND	0	100	Best-effort background

Aging prevents starvation. A waiting job's effective priority rises with how long it has waited ($\text{base} + \text{waitedSec} \times 150$), so a steady stream of foreground requests can't indefinitely block background work.

```
effective = priority.base + (waitedSeconds * 150) // highest effective wins
```

NON-PREEMPTIVE, ON PURPOSE

A running generation finishes before the next is admitted — there is no mid-generation pause/resume. True token-boundary preemption needs multiple KV sequences decoding concurrently (the continuous-batching upgrade). For a single warm context it would be dishonest to claim otherwise; generations are seconds long, so head-of-line blocking is bounded.

■ REFERENCES

Scheduler source: `runtime-service/.../service/AIScheduler.kt` (internal) · arch doc Part 08

Priority + aging: [Aging \(scheduling\) — background](#) (external)

Continuous batching: [Continuous batching for LLM serving \(Anyscale\)](#) (external)

SECTION 09

Speculative decoding

Implemented, verified — and deliberately GPU-only.

Speculative decoding runs a small, same-tokenizer **draft** model to propose N tokens, then has the big **target** model verify them in a single batched forward pass; accepted tokens are kept. When verification is near-free, you get several tokens for roughly the cost of one. A `ModelSpec` can declare a `draftId`, and `attachDraft()` wires it in.

WHY IT'S GATED TO GPU BACKENDS

The speedup depends on batched verification of N draft tokens being ~free — true on a bandwidth-bound GPU, but **not on CPU**, where a batch of N costs ~N× the matmul work. Measured here: acceptance was a healthy 50–60%, yet end-to-end it ran **~40–55% slower** than plain decode, because the 1B draft is only ~2× cheaper than the 3B target (not 3×) and CPU verify isn't free. So it's **gated to GPU backends only**; the code stays for when the Vulkan backend ships.

PROJECT MEMORY

Don't re-enable speculative decoding on CPU expecting a speedup — it's a documented net loss there. See `EdgeLMService.attachDraftLocked()` and `MODEL-OPTIMIZATION-FINDINGS.md`.

■ REFERENCES

Draft wiring: `NativeBridge.attachDraft` / `EdgeLMService.attachDraftLocked` (internal)

Original method: [Fast Inference via Speculative Decoding \(Leviathan et al., 2023\)](#) (external)

In llama.cpp: [llama.cpp speculative example](#) (external)

SECTION 10

The two front doors

A native Binder SDK and an OpenAI-compatible HTTP shim.

The same engine is reachable two ways, chosen by what the caller already speaks.

1 • Binder / AIDL (the SDK path)

First-class, efficient, and identity-aware: Binder carries the caller UID, so the runtime knows exactly which app is asking and can apply permissions and scheduling per app. This is the path the SDK's `EdgeLM.chat()` uses.

2 • OpenAI-compatible HTTP shim

A NanoHTTPD server bound to `127.0.0.1:1408`, inside the service process, so existing OpenAI SDKs / cURL work unmodified by pointing `base_url` at `http://localhost:1408/v1`.

```
POST /v1/chat/completions (stream=true -> SSE, else one JSON object)
GET /v1/models
GET /health
```

SPIKE-LEVEL SECURITY ON THE SHIM

The HTTP shim is loopback-only, has no auth, and — unlike Binder — loses the caller's app identity. It's a localhost dev convenience and is enabled only in debug builds; release builds are Binder-only. A later permission broker gates it properly.

■ REFERENCES

HTTP shim source: `runtime-service/.../service/EdgeLMHttpServer.kt` (internal) · arch doc Part 05

API shape: [OpenAI Chat Completions API reference](#) (external)

Streaming: [Server-Sent Events \(MDN\)](#) (external)

SECTION 11

Security & permission model

Mutually-untrusting apps over shared silicon.

EdgeLM sits between apps that don't trust each other and a resource that touches the user's most sensitive data (their prompts) and most contended hardware (RAM, GPU, battery). The model must stop one app from reading another's prompts, monopolizing the SoC, or draining the battery — while reusing Android's existing machinery rather than inventing a parallel one.

HOW IT'S ENFORCED

Capability as a permission — using the runtime is a declared permission (`ai.edgelm.permission.USE_RUNTIME`), granted through the standard Android consent flow. The SDK declares it (and the package `<queries>`) via manifest merge, so apps get it automatically.

Identity via Binder — the Binder path carries the caller UID, so requests are attributable and per-app policy is possible.

Isolation — the runtime is its own process/sandbox; prompts are processed in memory and never persisted or sent anywhere.

The localhost HTTP shim is the explicit exception (no UID, no auth) and is therefore debug-only until a permission broker gates it — see Section 10.

■ REFERENCES

Permission wiring: sdk manifest (`USE_RUNTIME`, `queries`) (internal) · arch doc Part 07

Android permissions: [Permissions on Android — developer guide](#) (external)

Custom permissions: [Define a custom app permission](#) (external)

Privacy policy: docs/play/PRIVACY.md (internal)

SECTION 12

The developer SDK

On-device AI in a few lines; no weights to ship.

The SDK's whole purpose is that an app author never sees Binder, AIDL, mmap or llama.cpp. They add one dependency and collect a cold Flow<String> of tokens; all the heavy lifting lives in the runtime app.

1 · Add the dependency (JitPack)

```
// settings.gradle.kts
repositories { maven { url = uri("https://jitpack.io") } }

// app/build.gradle.kts
implementation("com.github.Chandra-Mauli-Sharma.EdgeLM:sdk:0.1.0")
```

2 · Ensure the runtime is installed, then stream

```
when (EdgeLM.status(context)) {
    Status.NOT_INSTALLED -> EdgeLM.promptInstall(context) // -> Play
    Status.AVAILABLE     -> EdgeLM.initialize(context)    // bind service
}

EdgeLM.chat(model = "default", prompt = "Hello",
            sessionId = "chat-1", priority = EdgeLM.INTERACTIVE)
    .collect { token -> appendToUi(token) }
```

The Flow is **cold**: work starts on collect, and cancelling the collector's scope cancels the on-device decode. If the runtime is missing or unresponsive the SDK throws EdgeLMUnavailableException instead of hanging, so the app can fall back or prompt install. initialize() is idempotent; shutdown() unbinds but leaves the runtime alive for other apps.

■ REFERENCES

SDK + guide: [sdk/.../EdgeLM.kt](#) · [docs/INTEGRATION.md](#) · [samples/hello-edgelm](#) (internal)

Kotlin Flow: [Kotlin Flow — official docs](#) (external)

Distribution: [JitPack — build from a git tag](#) (external)

SECTION 13

Glossary & reference index

Fast lookup for the terms above.

GGUF	Single-file model format used by llama.cpp; holds weights + metadata, designed for mmap.
Q4_0	4-bit quantization that llama.cpp repacks into fast i8mm/dotprod ARM kernels at load.
imatrix	Importance-matrix calibration: weights which parameters matter during quantization.
mmap	Map a file into address space so the kernel page cache backs the model; zero-copy, shareable.
Binder / AIDL	Android's kernel IPC and its interface language; the identity-aware SDK transport.
oneway	AIDL modifier for fire-and-forget calls, used so token callbacks never block the engine.
KV cache	Cached attention keys/values; enables cheap multi-turn when a sessionId is reused.
Prefill vs decode	Prefill reads the prompt (compute-bound); decode emits tokens (bandwidth-bound).
Aging	Scheduling technique that raises a waiting job's priority over time to prevent starvation.
Speculative decoding	Draft proposes, target verifies in a batch; a win on GPU, a loss on CPU here.
Foreground service	Android service kept alive/visible so the shared runtime survives OEM freezers.
JNI	Java Native Interface; the NativeBridge seam between Kotlin and native llama.cpp.

PRIMARY INTERNAL SOURCES

README.md · docs/INTEGRATION.md · docs/EdgeLM-Architecture.pdf (Parts 01–16) · docs/MODEL-OPTIMIZATION-FINDINGS.md · contract/*.aidl · runtime-service service/* · runtime/ModelCatalog.kt · sdk/EdgeLM.kt · samples/hello-edgelm

Repository: github.com/Chandra-Mauli-Sharma/EdgeLM · Play: [EdgeLM Runtime](#)